



Report writing for Artificial intelligent

Assignment 2

Asra Anees

2022-09-10

Task 1:

Travelling sales man problem

The TSP is a classic problem in optimization, operation research and mathematic. The problem defines suppose the person visit separate different cities just once and came back to the original city. What is the shortest path to visit all the routes. The genetic algorithm is used to solve this problem by applying fitness function, crossover and mutation.

Genetic Algorithm

GA solve the problem by emulating evolution of genetic process. The process involves natural evaluation of chromosomes, fitness function, crossover and mutation.

The algorithm initializes with randomly generated chromosomes known as population. Each chromosomes represent the different solution. The initializes chromosomes goes through the process of crossover and mutation to create their new offspring. The fitness function applies to detect the best value to become a next generation. The evaluation process repeats until the iteration threshold reached.

TSP solves by GA

According to the given program there is 21 cities. Double array is generating the of 21 sizes. The 1st array represents the random number and 2nd the define the chromosomes.

Crossover can change the pattern to create the new offspring it is true until the cities is not repeated. mutation can change in between the pattern which gives another new value.

Solution of the given task

Population size = 5

Mutation rate = 0.9, 0.6, 0.3, 0.3

mutation									
population size 5		0.9		0.6		0.3		0.1	
		Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution
	1	2162	611.6312	8366	673.9878679	7024	517.9176896	2451	716.0867699
	2	4407	545.4130166	7459	514.6124863	5227	609.2347905	6632	618.0407812
	3	3192	561.6468868	5926	505.2102645	4286	670.4647224	3001	710.7796509
	4	7272	603.0372059	8401	650.6508387	6024	552.8773389	3015	706.9314203
	5	1529	631.1255034	1090	644.9216242	430	624.9159723	9755	513.6834219
	6	5136	648.2784891	4909	614.0554495	51	599.6865667	477	543.81745
	7	4908	641.4803943	1132	666.0217153	8636	699.673665	7662	644.6905253
	8	4036	679.2732103	227	658.8648337	1995	620.0564413	9372	655.8633292
	9	4600	593.1492947	8880	500.6966299	68	561.7090243	1957	684.3756471
	10	1956	582.667214	2133	626.5651641	4011	633.9436489	1325	676.5200612
Average			609.7702415		605.5586874		609.047986		647.0789057

Population size = 10

Mutation rate = 0.9, 0.6, 0.3, 0.3

mutation									
population size 10		0.9		0.6		0.3		0.1	
		Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution
	1	4054	549.71262	7178	582.2744739	726	619.6574408	7182	631.3435777
	2	9440	583.68967	6541	568.9748873	2214	581.2283447	4921	661.9071425
	3	4350	612.89026	3227	571.1572697	4739	658.4402801	3405	596.3643422
	4	5830	612.09339	7183	712.4643157	3382	583.0578592	7916	664.4466749
	5	74	593.14075	6415	630.3808622	5907	657.8683442	4373	596.3343921
	6	3530	543.74742	4446	491.2188017	746	570.4444535	7351	607.902483

	7	3854	553.6042	9807	603.6098217	33	623.8341566	174	566.6494105
	8	7195	609.14664	7654	609.4765879	2291	648.1902673	2375	507.0102687
	9	6180	586.18158	3876	645.6533732	5889	540.0536249	1547	516.3289715
	10	2426	660.26616	4589	586.2177988	733	499.1784982	6662	670.4295521
Average			590.44727		600.1428192		598.1953269		601.8716815

Population size = 20

Mutation rate = 0.9, 0.6, 0.3, 0.3

mutation									
population size 20		0.9		0.6		0.3		0.1	
		Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution
	1	1021	664.6689676	6130	638.2204774	4489	579.8699474	9104	533.4452509
	2	1284	578.4086128	7743	622.0349762	3735	580.3279628	1473	584.4112703
	3	5381	652.5927169	2738	684.7036306	1003	556.4088582	8185	561.7741154
	4	8288	595.7460534	3165	517.6283857	7590	485.3498498	7289	484.5067406
	5	5809	615.8019792	2742	635.8982931	5197	588.7826743	8364	592.1559658
	6	364	548.0522438	6757	657.6750703	2425	601.279627	3440	668.4652576
	7	6174	597.04892	9110	626.3413705	9432	570.4155456	4201	585.1273375
	8	4996	558.9405313	6693	580.64552	7424	664.3440581	803	615.4867392
	9	1648	565.056938	7136	580.9968987	9145	607.2267774	7879	557.7744795
	10	8291	509.8541153	1134	607.8286351	5805	578.2442507	4371	638.9490795
Average			588.6171078		615.1973258		581.2249551		582.2096237

Population size = 50

Mutation rate = 0.9, 0.6, 0.3, 0.3

mutation									
population size 50		0.9		0.6		0.3		0.1	
		Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution
	1	4539	524.3341304	7548	562.4927162	5211	579.2015026	3851	519.7670904
	2	5808	523.2992024	6086	532.948916	4339	585.8383686	2443	545.5358155
	3	1380	439.7695342	6946	531.3486901	7599	605.1293206	4173	482.773337
	4	1681	565.6071041	2169	639.9297342	2534	566.1915756	8452	524.8559819
	5	4482	583.1407791	3998	524.6000195	1658	501.1915796	238	543.0493199
	6	7353	497.4539239	1468	583.5677926	4358	505.2948545	7681	536.2560063
	7	1401	527.1414476	2910	550.7151379	256	600.8189396	4394	562.9102143
	8	2345	598.6079827	5211	561.683115	9684	637.6160637	4238	584.9651135
	9	1005	575.0866177	6525	517.8181397	5750	597.2028212	9634	617.4465315
	10	9072	534.3818973	5466	537.359794	1788	463.7813058	8187	518.0325182
Average			536.8822619		554.2464055		564.2266332		543.5591928

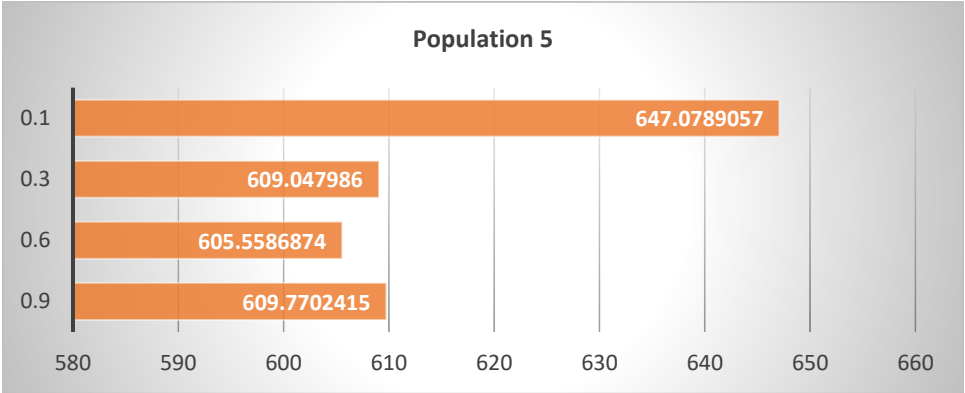
Population size = 100

Mutation rate = 0.9, 0.6, 0.3, 0.3

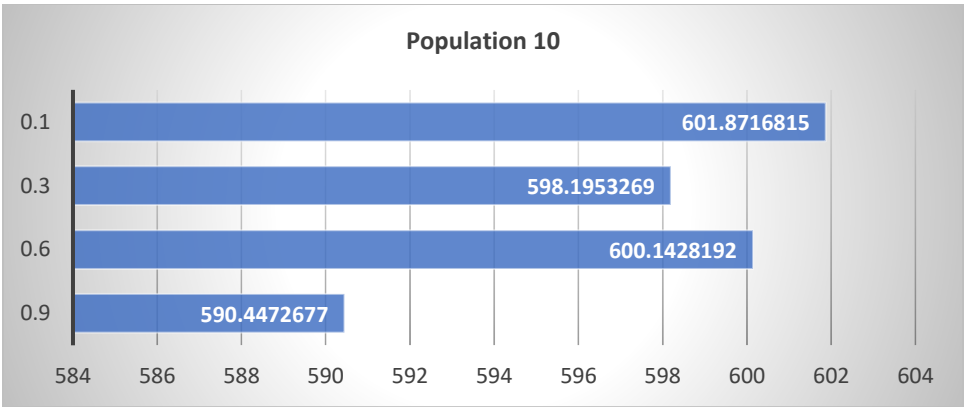
mutation									
population size 20		0.9		0.6		0.3		0.1	
		Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution	Iteration	Best Solution
	1	1592	532.0191995	9208	617.6641457	1464	608.4714441	5906	579.7484404
	2	7792	587.8214524	7678	598.6223144	2675	532.327574	6969	512.9902129

	3	681	496.0273327	2358	641.0298684	8855	510.3406534	9315	474.0980997
	4	9079	481.7528532	3273	496.5522649	3789	507.9949112	8130	563.7315357
	5	5087	580.0026943	9590	548.9336501	3523	577.9395111	9743	482.6761839
	6	2611	518.7401174	2997	592.6792451	6076	525.9476312	7899	518.3492227
	7	5524	485.0003846	711	599.7103685	7289	538.0219882	6447	568.1842633
	8	5514	590.5352865	7785	533.1609136	7903	618.2779886	922	526.1819014
	9	8290	522.9045908	9064	554.6353148	1583	620.7392456	3278	622.6308162
	10	3969	581.7835815	3022	516.4146207	7493	512.3839778	9910	497.1053198
Average			537.6587493		569.9402706		555.2444925		534.5695996

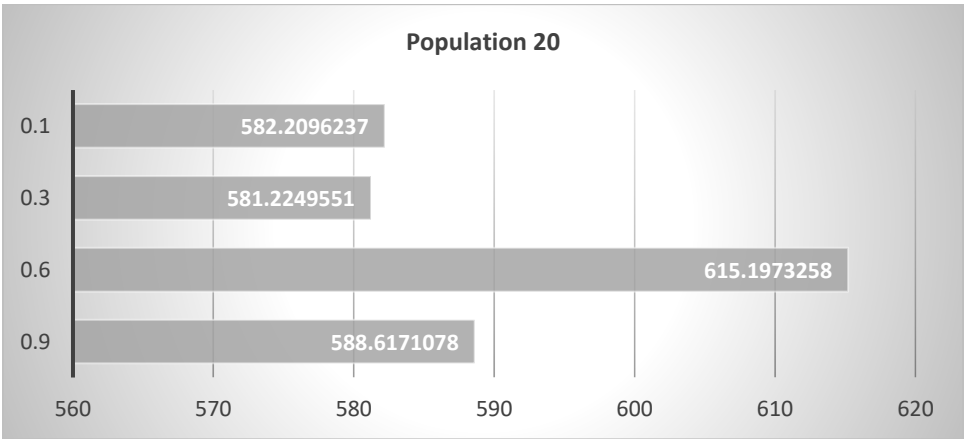
The average of population 5 with mutation 0.9, 0.6, 0.3, 0.1



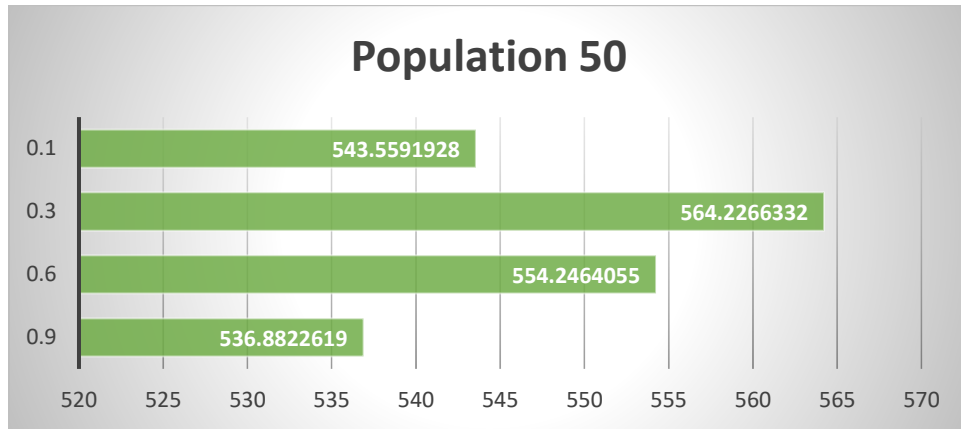
The average of population 10 with mutation 0.9, 0.6, 0.3, 0.1



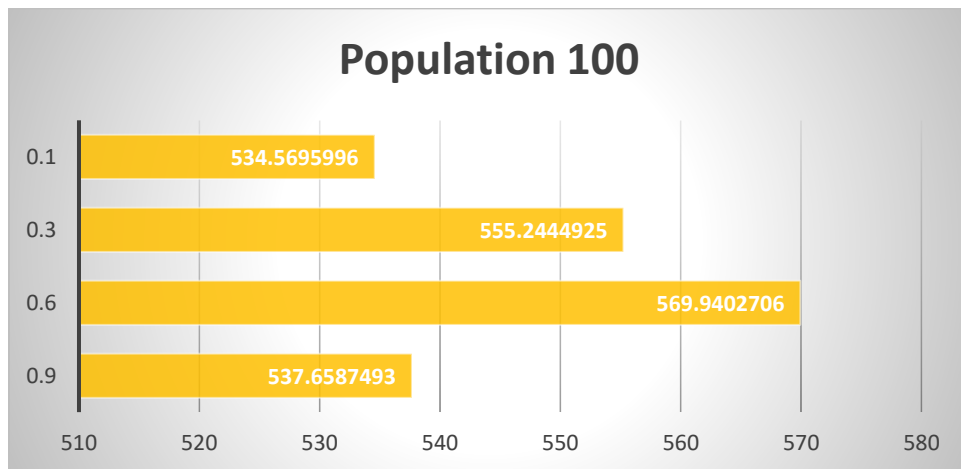
The average of population 20 with mutation 0.9, 0.6, 0.3, 0.1



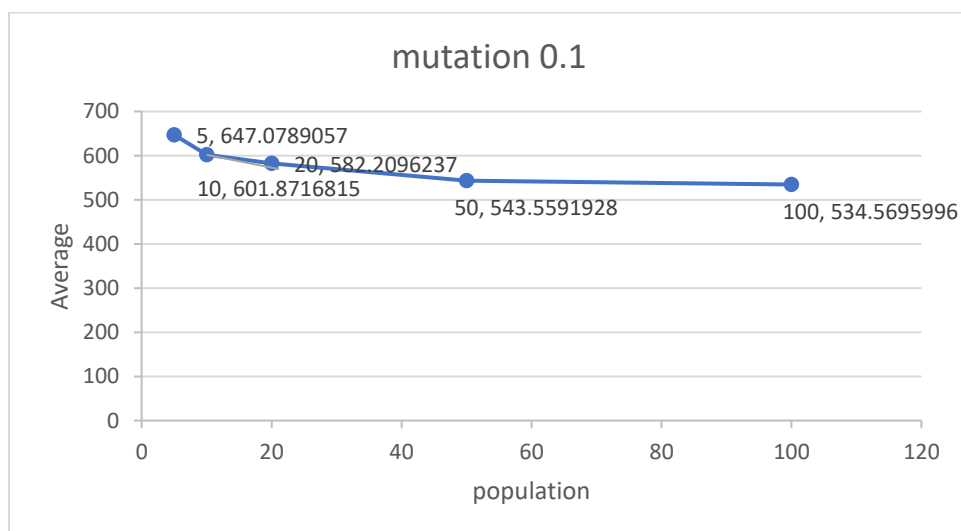
The average of population 50 with mutation 0.9, 0.6, 0.3, 0.1



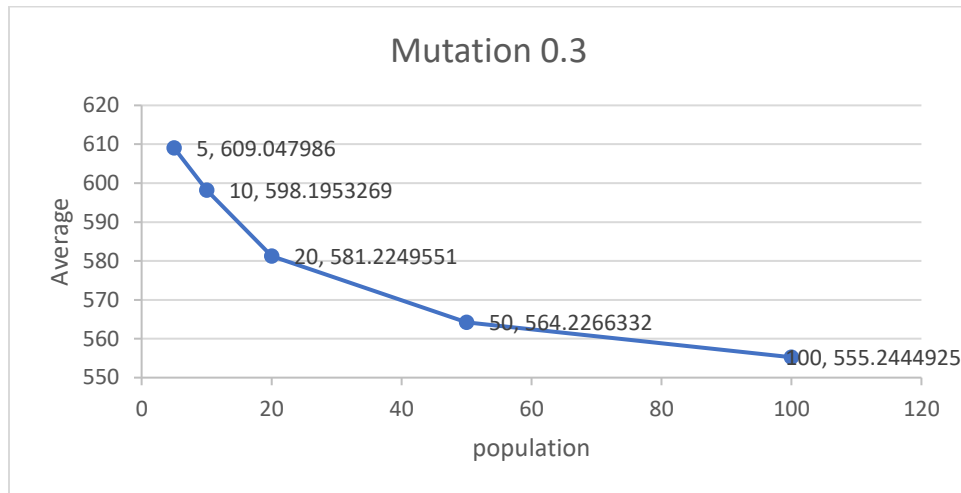
The average of population 100 with mutation 0.9, 0.6, 0.3, 0.1



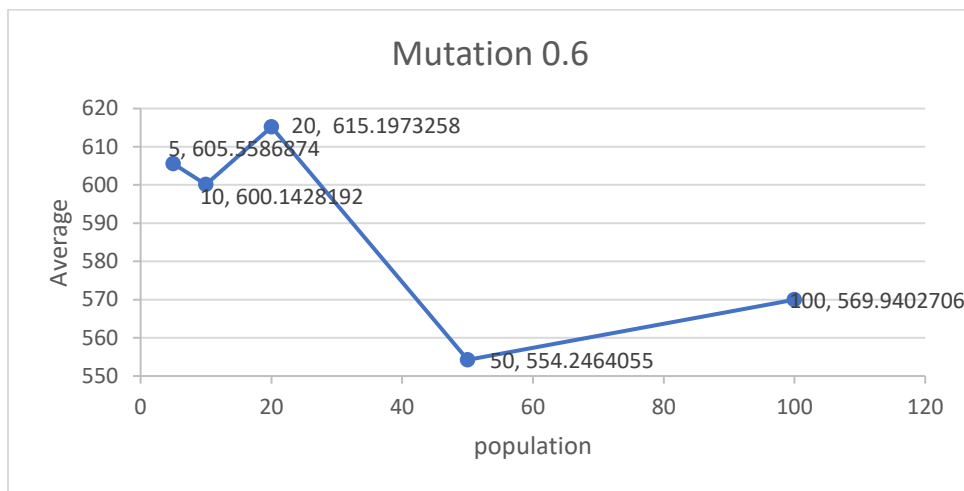
The average of mutation 0.1 with population 5, 10, 20, 50, 100



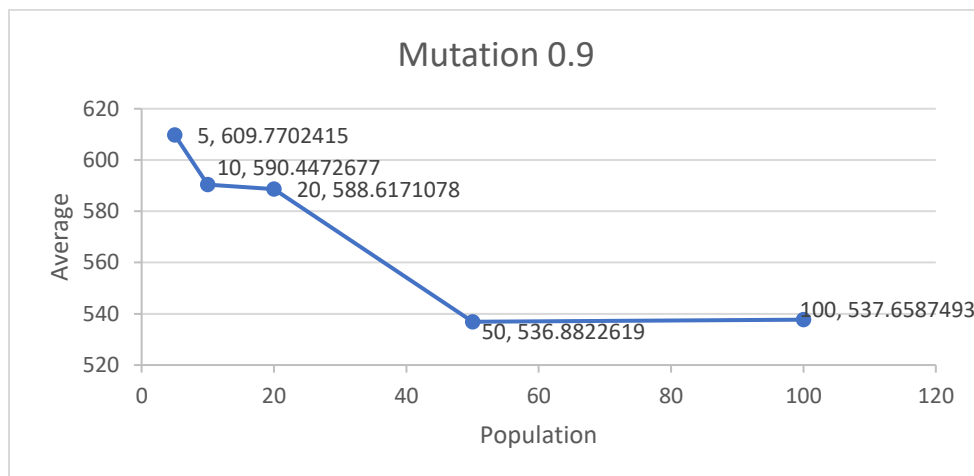
The average of mutation 0.3 with population 5, 10, 20, 50, 100



The average of mutation 0.6 with population 5, 10, 20, 50, 100



The average of mutation 0.9 with population 5, 10, 20, 50, 100



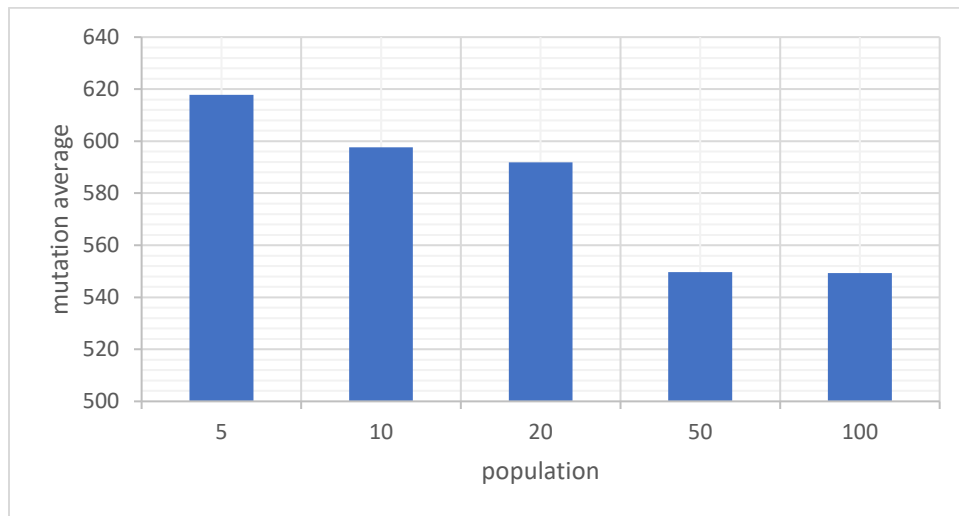
Result

The TSP solved by genetic algorithm finding the shortest path to visits the 20 cities. The fitness function defines the best solution. Experiment is done by running the algorithm 10 different time by changing the value of population and mutation the results is show at above given table and graph. The average value is increase when the population size is 5 with mutation 0.1, 0.3, 0.6 and 0.9 but the average value is decrease at population 100

and the time complexity of algorithm increase each time running algorithm takes 6 to 7 min to calculate the best solution at population size 100. The below table shows the average of population size 5 with all mutation is grater then all other population and population size 100 have smaller. The mutation rate at 0.6 have greater value with all population

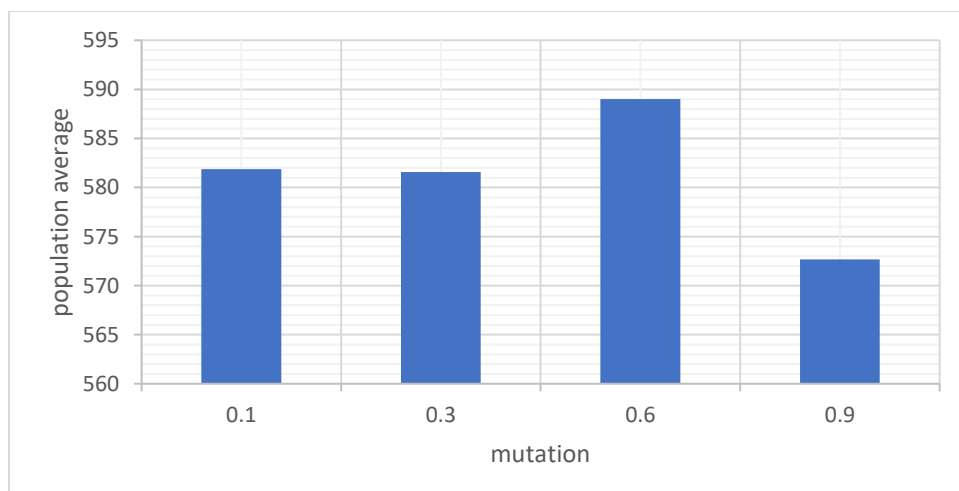
The average of all population with all mutation

population	Mutation Average (0.1, 0.3, 0.6, 0.9)
5	617.8639552
10	597.6642738
20	591.8122531
50	549.7286234
100	549.353278
Total average	581.2844767



The average of all mutation with all population

Mutation	Population Average (5, 10, 20, 50, 100)
0.1	581.8578007
0.3	581.5878787
0.6	589.0171017
0.9	572.6751257
Total average	581.2844767



Task 2

The fitness function gives the best solution. For maximization the mathematical equation is applied with higher rate of population which take higher cost for implementation. Thus, it is non-deterministic algorithm that why it gives the different as output at every time when the program is run.

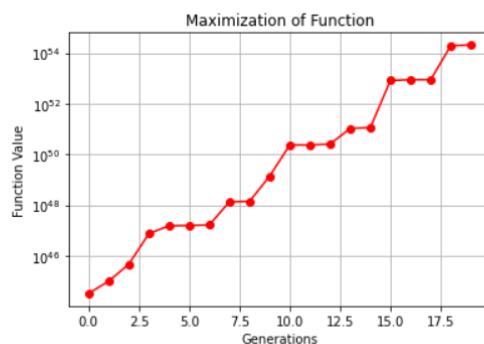
```
In [142]: def fitness(p):  
# Evaluating fitness Interference function "double fit (doublep[])".  
fitness=np.zeros((len(p),1))  
for i in range(len(p)):  
x,y,z = p[i][0] , p[i][1] , p[i][2]  
# Define your fitness function here  
fitness[i,0] = 2*x*z*np.exp(-x) - 2*y**3 + y**2 - 3*z**3  
return fitness
```

The output by applying the mathematical equation to solve maximization problem

$$2xz \exp(-x) - 2y^3 + y^2 - 3z^3$$

The output in graph

```
In [82]: import matplotlib.pyplot as plt  
Evaluations = Scores  
plt.plot(Evaluations,'o-',color='red')  
plt.yscale('log')  
plt.xlabel('Generations')  
plt.ylabel('Function Value')  
plt.title("Maximization of Function")  
plt.grid(True)  
plt.show()
```



Task 2 program python file is submitted by name "Task2.ipynb"

Task 3

Maze Game path finding problem solve by using genetic algorithm Code is submitted in folder name "Task3-Maze".

Program can be run by creating 2D array size of 20 x 33

[illegible]